Implementação dos Algoritmos de Busca em Largura (BFS) e Busca em Profundidade (DFS)

Neste notebook, vamos implementar dois algoritmos clássicos de busca em grafos: Busca em Largura (Breadth-First Search - BFS) e Busca em Profundidade (Depth-First Search - DFS).

O objetivo é utilizá-los para encontrar um caminho entre duas cidades em um mapa representado como um grafo, onde os nós são as cidades e as arestas representam as conexões diretas entre elas (como estradas).

Esse tipo de problema é comum em áreas como inteligência artificial, sistemas de navegação, análise de redes e planejamento de rotas.

Instalação e Importação da Biblioteca para Visualização

Para facilitar o entendimento dos algoritmos de busca, vamos utilizar uma biblioteca externa para visualizar a árvore de busca gerada.

Observação: Esta biblioteca não é necessária para a execução dos algoritmos de busca. Ela serve apenas como um recurso visual e didático, para ajudar na compreensão do funcionamento das buscas.

```
!pip install treelib
from treelib import Node, Tree
```

```
Collecting treelib
  Downloading treelib-1.8.0-py3-none-any.whl.metadata (3.3 kB)
Requirement already satisfied: six>=1.13.0 in /usr/local/lib/python3.12/dist-packages (from treelib) (1.17.0)
Downloading treelib-1.8.0-py3-none-any.whl (30 kB)
Installing collected packages: treelib
Successfully installed treelib-1.8.0
```

Definição da Classe Estado

A classe Estado representa cada nó da árvore de busca gerada pelos algoritmos (BFS ou DFS). Cada estado contém informações essenciais sobre o ponto atual da busca.

Neste exemplo, cada estado é definido apenas por:

nome: o nome da cidade que o estado representa.

pai: uma referência ao estado anterior (ou seja, à cidade de onde se chegou ao estado atual).

Essa estrutura é fundamental para que, ao encontrar o estado objetivo (cidade de destino), seja possível reconstruir o caminho completo até ele. Isso é feito percorrendo os atributos pai de forma iterativa, do estado final até o estado inicial (a raiz da árvore).

Observação importante: Embora os algoritmos de busca trabalhem diretamente sobre um grafo (e não construam a árvore completa), o uso do atributo pai permite reconstruir a árvore de busca parcial a partir de qualquer nó da fronteira até a raiz.

```
class Estado:
  def __init__(self, cidade, pai):
    self.cidade = cidade
    self.pai = pai
```

Definição da Classe Busca

A classe Busca é a classe base (abstrata) que define a estrutura comum para os algoritmos de busca. A partir dela, serão derivadas as classes específicas como BuscaLargura (BFS) e BuscaProfundidade (DFS).

Essa classe não executa a busca por si só — ela fornece os elementos fundamentais que serão utilizados pelas subclasses.

Principais Atributos:

visitados (set) Um conjunto que armazena os nomes das cidades que já foram visitadas durante a busca. Isso evita ciclos e visitas repetidas — essencial quando a busca é feita em um grafo, e não em uma árvore.

árvore (list) Lista usada apenas para armazenar os vértices visitados, com o propósito de visualização ou análise posterior. Ela não interfere no funcionamento da busca.

rotas (dict) Representa o grafo do mapa de cidades, por meio de uma lista de adjacências implementada como um dicionário. Cada chave do dicionário é uma cidade, e o valor é uma lista com as cidades vizinhas diretamente conectadas.

```
class Busca:

  def __init__(self):
    self.rotas = {'Porto Alegre': ['Florianópolis','São Paulo'],
                  'Florianópolis': ['Curitiba','Porto Alegre'],
                  'Curitiba': ['Florianópolis','São Paulo','Rio de Janeiro'],
                  'São Paulo':['Belo Horizonte', 'Curitiba', 'Porto Alegre', 'Salvador'],
                  'Rio de Janeiro':['Belo Horizonte','Cuiabá', 'Curitiba'],
                  'Belo Horizonte': ['Brasília','Cuiabá', 'São Paulo','Rio de Janeiro'],
                  'Brasília': ['Belo Horizonte','Fortaleza'],
                  'Salvador': ['Fortaleza','São Paulo'],
                  'Cuiabá': ['Belo Horizonte','Manaus','Rio de Janeiro'],
                  'Fortaleza': ['Manaus','Salvador','Brasília'],
                  'Manaus': ['Cuiabá','Fortaleza']
  }

# Implementação do método que realiza a busca efetivamente. Este método recebe como parâmetros as cidades de origem e de destin
# a estrura de dados que armazena a fronteira (os nós ainda não expandidos), o conjunto que armazena os nomes das cidades já vi
# e o objeto que vai armazenar a árvore gerada
  def busca(self, origem, destino, fronteira):
    atual = Estado(origem, None)
    fronteira.put(atual)
    visitados = set()
    visitados.add(atual.cidade)
    qtdVisitados = 1
    qtdExpandidos = 0
    ##### As próximas linhas são desnecessárias caso não se deseje ver a árvore no final da execução
    arvore = Tree()
    arvore.create_node(atual.cidade, atual)
    #####
    resultado = None
    while not fronteira.empty() and resultado == None:
      atual = fronteira.get()
      qtdExpandidos += 1
      resultado, fronteira, visitados, qtdVisitados, arvore = self.geraFilhos(atual, destino, fronteira, visitados, qtdVisitadc
    return resultado, qtdVisitados, qtdExpandidos, arvore

#Implementação do método geraFilhos. Este método recebe um nodo da árvore como parâmetro e gera os filhos possíveis consultando
#Ele também recebe como parâmetro a cidade de destino para saber se chegou ao resultado ou não.
  def geraFilhos(self, atual,  destino, fronteira, visitados, qtdVisitados, arvore):
    cidades = self.rotas.get(atual.cidade)
    for c in cidades:
      if (c==destino):
        qtdVisitados += 1
        novo = Estado(c, atual)
        visitados.add(c)
        # Esta linha é desnecessária caso não se deseje ver a árvore no final da execução
        arvore.create_node(c, novo, parent=atual)
        return novo, fronteira, visitados, qtdVisitados, arvore
      else:
        # Se os dois ifs abaixo estiverem comentados, a busca vai gerar estados repetidos na árvore, inclusive filhos que serão
        # Se o primeiro if estiver comentado e o segundo if não a busca não vai gerar nenhum estado repetido no árvore
        # Se o segundo if estiver comentando e o primeiro não, a busca vai gerar estados repetidos na árvore, mas não gerará fi

        #if atual.pai == None or c != atual.pai.cidade:
        if c not in visitados:
          qtdVisitados += 1
          novo = Estado(c, atual)
          fronteira.put(novo)
          visitados.add(c)
```

```
            # Esta linha é desnecessária caso não se deseje ver a árvore no final da execução
            arvore.create_node(c, novo, parent=atual)

    return None, fronteira, visitados, qtdVisitados, arvore

#Implementação do método que apresenta os resultados da busca. O resultado está na ordem inversa pois ele é descoberto retornan
#consultando o atributo pai da classe Estado. Para apresentá-lo na ordem correta, basta inserir os estados em uma lista,
#em vez de mostrá-los diretamente e depois mostrar a lista na ordem inversa.
  def mostraResultado(self, resultado, qtdVisitados, qtdExpandidos, arvore):
    if (resultado==None):
      print('Solução não encontrada.')
    else:
      print('***Rota encontrada***')
      while (resultado != None):
        print(resultado.cidade)
        resultado = resultado.pai
    print('Estados visitados: ',qtdVisitados)
    print('Estados expandidos: ',qtdExpandidos)
    print('****Árvore gerada****')
    arvore.show()
```

Aqui termina a classe **Busca** e são definidas as classes derivadas que implementam cada um dos diferentes algoritmos de busca.

Definição da **classe BuscaLargura**. Os métodos de busca e geração dos filhos são os mesmos da classe base. O que caracteriza a Busca em Largura é a implementação da fronteira como uma Fila (FIFO).

```
# Importação da classe Queue que implementa a Fila usada para armazenar a fronteira
from queue import Queue

class BuscaLargura(Busca):

  def realizaBusca(self, origem, destino):
    fronteira = Queue()
    resultado, qtdVisitados, qtdExpandidos, arvore = self.busca(origem, destino, fronteira)
    self.mostraResultado(resultado, qtdVisitados, qtdExpandidos, arvore)
```

Como Executar a Busca em Largura (BFS)

Para realizar a busca em largura, siga os passos abaixo:

Instancie um objeto da classe **BuscaLargura.**

1. Item Instancie um objeto da classe **BuscaLargura.**
2. Item Defina as cidades de origem e destino.
3. Item Chame o método de busca passando as cidades escolhidas.

Dessa forma, o algoritmo irá encontrar um caminho da cidade de origem até a cidade de destino, explorando o grafo de forma ampla (nível por nível).

```
algbusca = BuscaLargura()
algbusca.realizaBusca('Curitiba','Salvador')
```

```
***Rota encontrada***
Salvador
São Paulo
Curitiba
Estados visitados:  7
Estados expandidos:  3
****Árvore gerada****
Curitiba
├── Florianópolis
│   └── Porto Alegre
├── Rio de Janeiro
└── São Paulo
    ├── Belo Horizonte
    └── Salvador
```

Definição da Classe BuscaProfundidade

A classe **BuscaProfundidade** herda os métodos de busca e geração dos filhos da classe base Busca.

O que diferencia a busca em profundidade é a forma como a fronteira é implementada: como uma Pilha (LIFO).

Isso faz com que o algoritmo explore o máximo possível ao longo de um caminho antes de retroceder e tentar outros caminhos.

```
# Importação da classe LifoQueue que implementa a Pilha usada para armazenar a fronteira
from queue import LifoQueue

class BuscaProfundidade(Busca):

  def realizaBusca(self, origem, destino):
    fronteira = LifoQueue()
    resultado, qtdVisitados, qtdExpandidos, arvore = self.busca(origem, destino, fronteira)
    self.mostraResultado(resultado, qtdVisitados, qtdExpandidos, arvore)
```

Como Executar a Busca em Profundidade (DFS)

Para realizar a busca em profundidade, siga estes passos:

1. Item Crie uma instância da classe BuscaProfundidade.
2. Item Defina as cidades de origem e destino.
3. Item Execute o método de busca passando esses parâmetros.

Assim, o algoritmo vai explorar os caminhos profundamente, indo até o máximo possível antes de voltar e tentar outras rotas.

```
algbusca = BuscaProfundidade()
algbusca.realizaBusca('Curitiba','Fortaleza')
```

```
***Rota encontrada***
Fortaleza
Manaus
Cuiabá
Rio de Janeiro
Curitiba
Estados visitados:  8
Estados expandidos:  4
****Árvore gerada****
Curitiba
├── Florianópolis
├── Rio de Janeiro
│   ├── Belo Horizonte
│   └── Cuiabá
│       └── Manaus
│           └── Fortaleza
└── São Paulo
```